

ANSI SQL Using MySQL

Module 2 — Exercises & Solutions

Event Management Portal Schema

25 Solved Exercises, with Query Output

Exercise 1: User Upcoming Events

Show a list of all upcoming events a user is registered for in their city, sorted by date.

```
SELECT
    u.user_id,
    u.full_name,
    e.event_id,
    e.title,
    e.city,
    e.start_date
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
    AND e.city = u.city
ORDER BY u.user_id, e.start_date;
```

OUTPUT

Result Grid					
user_id	full_name	event_id	title	city	start_date
1	Alice Johnson	1	Tech Innovators Meetup	New York	2025-06-10 10:00:00
5	Ethan Hunt	3	Frontend Development Bootcamp	Los Angeles	2025-07-01 10:00:00
2 rows returned					

Exercise 2: Top Rated Events

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
SELECT
    e.event_id,
    e.title,
    AVG(f.rating) AS avg_rating,
    COUNT(f.feedback_id) AS feedback_count
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

OUTPUT

Result Grid			
event_id	title	avg_rating	feedback_count
Empty set (0 rows returned)			
0 rows returned			

Exercise 3: Inactive Users

Retrieve users who have not registered for any events in the last 90 days.

```
SELECT u.*
FROM Users u
WHERE u.user_id NOT IN (
    SELECT r.user_id
    FROM Registrations r
    WHERE r.registration_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
);
```

OUTPUT

Result Grid				
user_id	full_name	email	city	registration_date
1	Alice Johnson	alice@example.com	New York	2024-12-01
2	Bob Smith	bob@example.com	Los Angeles	2024-12-05
3	Charlie Lee	charlie@example.com	Chicago	2024-12-10
4	Diana King	diana@example.com	New York	2025-01-15
4 rows returned				

Exercise 4: Peak Session Hours

Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```
SELECT
    event_id,
    COUNT(*) AS session_count
FROM Sessions
WHERE TIME(start_time) >= '10:00:00'
    AND TIME(start_time) <= '12:00:00'
GROUP BY event_id;
```

OUTPUT

Result Grid	
event_id	session_count
1	2
3	1
2 rows returned	

Exercise 5: Most Active Cities

List the top 5 cities with the highest number of distinct user registrations.

```
SELECT
```

```

    u.city,
    COUNT(DISTINCT r.user_id) AS distinct_users
FROM Registrations r
JOIN Users u ON r.user_id = u.user_id
GROUP BY u.city
ORDER BY distinct_users DESC
LIMIT 5;

```

OUTPUT

Result Grid	
city	distinct_users
New York	2
Los Angeles	2
Chicago	1
3 rows returned	

Exercise 6: Event Resource Summary

Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

```

SELECT
    e.event_id,
    e.title,
    SUM(CASE WHEN res.resource_type = 'pdf' THEN 1 ELSE 0 END) AS pdf_count,
    SUM(CASE WHEN res.resource_type = 'image' THEN 1 ELSE 0 END) AS image_count,
    SUM(CASE WHEN res.resource_type = 'link' THEN 1 ELSE 0 END) AS link_count,
    COUNT(res.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources res ON e.event_id = res.event_id
GROUP BY e.event_id, e.title
ORDER BY e.event_id;

```

OUTPUT

Result Grid					
event_id	title	pdf_count	image_count	link_count	total_resources
1	Tech Innovators Meetup	1	0	0	1
2	AI & ML Conference	0	1	0	1
3	Frontend Development Bootcamp	0	0	1	1
3 rows returned					

Exercise 7: Low Feedback Alerts

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```

SELECT

```

```

    u.user_id,
    u.full_name,
    e.title AS event_name,
    f.rating,
    f.comments
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3;

```

OUTPUT

Result Grid				
user_id	full_name	event_name	rating	comments
Empty set (0 rows returned)				
0 rows returned				

Exercise 8: Sessions per Upcoming Event

Display all upcoming events with the count of sessions scheduled for them.

```

SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;

```

OUTPUT

Result Grid		
event_id	title	session_count
1	Tech Innovators Meetup	2
3	Frontend Development Bootcamp	1
2 rows returned		

Exercise 9: Organizer Event Summary

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```

SELECT
    o.user_id AS organizer_id,
    o.full_name AS organizer_name,
    e.status,
    COUNT(*) AS event_count

```

```

FROM Events e
JOIN Users o ON e.organizer_id = o.user_id
GROUP BY o.user_id, o.full_name, e.status
ORDER BY o.user_id, e.status;

```

OUTPUT

Result Grid			
organizer_id	organizer_name	status	event_count
1	Alice Johnson	upcoming	1
2	Bob Smith	upcoming	1
3	Charlie Lee	completed	1
3 rows returned			

Exercise 10: Feedback Gap

Identify events that had registrations but received no feedback at all.

```

SELECT DISTINCT e.event_id, e.title
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE f.feedback_id IS NULL;

```

OUTPUT

Result Grid	
event_id	title
3	Frontend Development Bootcamp
1 row returned	

Exercise 11: Daily New User Count

Find the number of users who registered each day in the last 7 days.

```

SELECT
    registration_date,
    COUNT(*) AS new_users
FROM Users
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)
GROUP BY registration_date
ORDER BY registration_date;

```

OUTPUT

Result Grid	
registration_date	new_users
Empty set (0 rows returned)	
0 rows returned	

Exercise 12: Event with Maximum Sessions

List the event(s) with the highest number of sessions.

```

SELECT
    e.event_id,
    e.title,
    s.session_count
FROM (
    SELECT event_id, COUNT(*) AS session_count
    FROM Sessions
    GROUP BY event_id
) s
JOIN Events e ON e.event_id = s.event_id
WHERE s.session_count = (
    SELECT MAX(cnt)
    FROM (SELECT COUNT(*) AS cnt FROM Sessions GROUP BY event_id) t
);

```

OUTPUT

Result Grid		
event_id	title	session_count
1	Tech Innovators Meetup	2
1 row returned		

Exercise 13: Average Rating per City

Calculate the average feedback rating of events conducted in each city.

```

SELECT
    e.city,
    ROUND(AVG(f.rating), 2) AS avg_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city
ORDER BY avg_rating DESC;

```

OUTPUT

Result Grid	
city	avg_rating
Chicago	4.5
New York	3.0
2 rows returned	

Exercise 14: Most Registered Events

List top 3 events based on the total number of user registrations.

```
SELECT
    e.event_id,
    e.title,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

OUTPUT

Result Grid		
event_id	title	total_registrations
1	Tech Innovators Meetup	2
2	AI & ML Conference	2
3	Frontend Development Bootcamp	1
3 rows returned		

Exercise 15: Event Session Time Conflict

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
SELECT
    a.event_id,
    a.session_id AS session_1_id,
    a.title AS session_1_title,
    b.session_id AS session_2_id,
    b.title AS session_2_title
FROM Sessions a
JOIN Sessions b
    ON a.event_id = b.event_id
    AND a.session_id < b.session_id
WHERE a.start_time < b.end_time
    AND b.start_time < a.end_time;
```

OUTPUT

Result Grid				
event_id	session_1_id	session_1_title	session_2_id	session_2_title
Empty set (0 rows returned)				
0 rows returned				

Exercise 16: Unregistered Active Users

Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT u.*
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
WHERE u.registration_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
AND r.registration_id IS NULL;
```

OUTPUT

Result Grid				
user_id	full_name	email	city	registration_date
Empty set (0 rows returned)				
0 rows returned				

Exercise 17: Multi-Session Speakers

Identify speakers who are handling more than one session across all events.

```
SELECT
    speaker_name,
    COUNT(*) AS session_count
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(*) > 1;
```

OUTPUT

Result Grid	
speaker_name	session_count
Empty set (0 rows returned)	
0 rows returned	

Exercise 18: Resource Availability Check

List all events that do not have any resources uploaded.

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Resources res ON e.event_id = res.event_id
WHERE res.resource_id IS NULL;
```

OUTPUT

Result Grid	
event_id	title
Empty set (0 rows)	
0 rows returned	

Exercise 19: Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

```
SELECT
    e.event_id,
    e.title,
    COUNT(DISTINCT r.registration_id) AS total_registrations,
    ROUND(AVG(f.rating), 2) AS avg_rating
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;
```

OUTPUT

Result Grid			
event_id	title	total_registrations	avg_rating
2	AI & ML Conference	2	4.5
1 row returned			

Exercise 20: User Engagement Index

For each user, calculate how many events they attended and how many feedbacks they submitted.

```
SELECT
    u.user_id,
    u.full_name,
    COUNT(DISTINCT r.event_id) AS events_attended,
    COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
LEFT JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY u.user_id;
```

OUTPUT

Result Grid			
user_id	full_name	events_attended	feedbacks_submitted
1	Alice Johnson	1	0
2	Bob Smith	1	1
3	Charlie Lee	1	1
4	Diana King	1	1
5	Ethan Hunt	1	0
5 rows returned			

Exercise 21: Top Feedback Providers

List top 5 users who have submitted the most feedback entries.

```
SELECT
    u.user_id,
    u.full_name,
    COUNT(f.feedback_id) AS feedback_count
FROM Users u
JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY feedback_count DESC
LIMIT 5;
```

OUTPUT

Result Grid		
user_id	full_name	feedback_count
2	Bob Smith	1
3	Charlie Lee	1
4	Diana King	1
3 rows returned		

Exercise 22: Duplicate Registrations Check

Detect if a user has been registered more than once for the same event.

```
SELECT
    user_id,
    event_id,
    COUNT(*) AS registration_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

OUTPUT

Result Grid		
user_id	event_id	registration_count
Empty set (0 rows returned)		
0 rows returned		

Exercise 23: Registration Trends

Show a month-wise registration count trend over the past 12 months.

```
SELECT
    DATE_FORMAT(registration_date, '%Y-%m') AS registration_month,
    COUNT(*) AS registration_count
FROM Registrations
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY registration_month
ORDER BY registration_month;
```

OUTPUT

Result Grid	
registration_month	registration_count
2025-04	2
2025-05	2
2025-06	1
3 rows returned	

Exercise 24: Average Session Duration per Event

Compute the average duration (in minutes) of sessions in each event.

```
SELECT
    event_id,
    ROUND(AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)), 2) AS avg_duration_minutes
FROM Sessions
GROUP BY event_id;
```

OUTPUT

Result Grid	
event_id	avg_duration_minutes
1	60.0
2	90.0
3	120.0
3 rows returned	

Exercise 25: Events Without Sessions

List all events that currently have no sessions scheduled under them.

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL;
```

OUTPUT

Result Grid	
event_id	title
Empty set (0 rows)	
0 rows returned	